

MODERN DATABASE APPLICATIONS PROJECT REPORT

Redis-Based Real-Time Clickstream Analytics System

Section 1 — Title Page

Course: Modern Database Applications

Class: AI66B

Members: Phan Hai Dang - 11247270

Supervisor: Tran Quang Toan

Abstract:

This project focuses on building a real-time clickflow analysis system using Redis and FastAPI. The main purpose of the system is to collect user interaction events, process them quickly, and display analysis results in real time.

Redis was chosen as the primary database because it offers very fast in-memory data processing and supports useful data structures such as Hash and Sorted Set. These features make it suitable for handling a large volume of event updates with low response times.

The system is designed to collect click events, store event-related data, and generate live statistics for analysis. Compared to traditional relational databases, Redis simplifies the implementation of ranking and temporary session management.

The results show that the system can efficiently process events and provide near-instantaneous analysis updates.

Section 2 — Introduction

2.1 Project Overview

This project is titled **Redis-Based Real-Time Clickstream Analytics System**.

The purpose of this project is to build a lightweight system to collect and analyze user interaction events in real time. Processing this data is crucial for monitoring system activity and understanding user behavior.

The system was developed using FastAPI for the backend and Redis as the main analytics database.

The implementation combines two approaches:

- A lightweight in-memory event buffer used for simplified API testing and fast local development
- Redis-based analytics processing for event statistics and timeline management

The main objectives of this project are:

- Capture clickstream events efficiently
- Process event statistics in near real time
- Store event metadata using Redis data structures
- Provide analytical insights through API responses

This project focuses on showing how Redis can support event-driven systems with low latency and simple data processing logic.

2.2 RDBMS (SQL) Limitations in this Context

If a relational database such as MySQL or PostgreSQL were used for this project, several practical limitations would appear.

The current system processes clickstream events through lightweight API endpoints and updates event statistics continuously. This requires handling many small write operations efficiently.

Another limitation is real-time statistical aggregation. In this project, event counters are updated directly using Redis increment operations. If SQL databases were used, generating the same statistics would require either repeated aggregation queries or maintaining additional counter tables.

For example, obtaining event frequency data in SQL would require queries such as:

```
```sql
SELECT event_type, COUNT(*)
FROM events
GROUP BY event_type;
```
```

As event volume increases, these queries become more expensive.

The project also maintains event ordering through Redis Sorted Sets. Implementing similar ranking behavior in relational databases would require indexed sorting operations, which would increase complexity.

Because of these reasons, using PostgreSQL or MySQL would make the implementation more complex and less efficient for this specific real-time event-processing workload.

2.3 Rationale for Technology Choice

Redis was selected because its design fits the requirements of this project very well.

The first reason is speed. Redis stores data in memory, allowing very fast read and write operations. Since clickstream analytics requires frequent event updates, this provides better performance than disk-based systems.

The second reason is Redis data structures. This project uses Redis features such as: **Hashes, Sorted Sets, TTL Support,...**

Another important reason is implementation simplicity. Redis allows the system to focus on query-driven design rather than relational schema normalization. This makes the code easier to maintain and reduces database-side complexity.

Although the current project still includes a lightweight in-memory buffer for simplified event handling during development, Redis serves as the main database technology for analytical processing and demonstrates the practical benefits of NoSQL systems for real-time applications.

Section 3 — Method

This section explains the implementation approach of the system based on the actual project source code.

The project is divided into two main components:

1. A lightweight FastAPI event tracking layer
2. A Redis-based analytics service layer

The FastAPI layer is responsible for receiving clickstream events and temporarily storing them in memory.

The Redis analytics layer is implemented separately to support scalable event storage and statistical processing.

This design allows simple local testing while also demonstrating Redis-based NoSQL analytics techniques.

3.1 Data Modeling

The project uses two data storage approaches.

In-Memory Event Buffer

The main API currently stores incoming events in a Python list:

```
events = []
```

This buffer is used for lightweight testing and fast event inspection.

To avoid unlimited memory growth, the buffer keeps only the latest 20 events.

This is implemented using:

```
if len(events) > 20:
```

```
    events.pop(0)
```

Redis Analytics Storage

Redis is used as the analytical processing layer.

It provides persistent event statistics and ordered event tracking.

The Redis implementation is defined inside the `AnalyticsService` class.

3.1.1 Query-Driven Design Analysis

The system supports the following operations.

Event Collection

Incoming clickstream events are received through:

POST /track

Each event is submitted as JSON data.

Example:

```
{  
  "event_type": "click",  
  "element": "submit_button"  
}
```

The event is appended to the in-memory buffer.

Event Retrieval

Recent events can be retrieved through:

GET /analytics

This endpoint returns the current event buffer.

Redis Statistical Processing

The Redis service supports:

- Event detail storage
- Event timeline ordering
- Event frequency counting
- Statistical retrieval

3.1.2 Detailed Data Model

In-Memory Structure

The temporary event buffer stores raw event dictionaries.

Example:

Redis Key Naming Convention

| Key Pattern | Data Type | Purpose |
|-------------------|------------|-----------------------------------|
| event:{timestamp} | Hash | Stores event details |
| events_timeline | Sorted Set | Maintains event order |
| stats:event_types | Hash | Stores event frequency statistics |

Event Detail Storage

Each event is assigned a unique key:

event_id = f"event:{int(time.time() * 1000)}"

This ensures unique event identification.

Hash Storage

Event metadata is stored using Redis Hashes.

Implementation:

```
await redis_pool.hset(event_id, mapping=event_data)
```

This stores structured event information.

Sorted Set Timeline

Events are inserted into a chronological timeline.

Implementation:

```
await redis_pool.zadd("events_timeline", {event_id: time.time()})
```

The timestamp is used as the score.

This allows ordered retrieval.

Event Statistics

Event counters are updated using:

```
await redis_pool.hincrby(
    "stats:event_types",
    event_data["event_type"],
    1
)
```

This tracks how often each event type occurs.

3.2 Implementation & DB-Specific Queries

The backend is implemented using FastAPI.

Redis integration uses the asynchronous Redis client.

Connection setup is defined as:

```
redis_pool = redis.from_url(
    settings.redis_url,
```

```
    decode_responses=True
)
```

This enables non-blocking Redis communication.

3.2.1 System Architecture

The actual request flow is:

Frontend Interface

↓

FastAPI API Layer

↓

In-Memory Event Buffer

↓

Analytics API Response

The Redis analytical layer exists as a separate processing service:

Event Data

↓

AnalyticsService

↓

Redis Storage

↓

Statistical Retrieval

This architecture reflects the current development-stage implementation.

3.2.2 Core Code Snippets

Event Tracking Endpoint

```
@app.post("/track")
```

```
async def track(request: Request):
```

This endpoint receives incoming event data.

Event Buffering

```
events.append(data)
```

Stores events temporarily.

Buffer Size Control

```
if len(events) > 20:
```

```
    events.pop(0)
```

Maintains only recent events.

Redis Event Save

```
await redis_pool.hset(event_id, mapping=event_data)
```

Stores event metadata.

Timeline Update

```
await redis_pool.zadd("events_timeline", {event_id: time.time()})
```

Adds event ordering.

Statistical Update

```
await redis_pool.hincrby(  
    "stats:event_types",  
    event_data["event_type"],  
    1  
)
```

Updates event statistics.

3.3 Advanced Features Implementation

Although the project is lightweight, it includes several advanced concepts.

Asynchronous API Design

FastAPI asynchronous request handling supports efficient event processing.

Redis AsyncIO Integration

Redis communication is asynchronous.

This improves scalability potential.

Rolling Event Window

The system limits memory usage by storing only the latest 20 events.

This creates an efficient temporary event cache.

Redis Analytical Layer

The separated analytics service demonstrates scalable event-processing design.

Although not fully integrated into the active request flow, it provides the foundation for future expansion.

Frontend Integration

The application serves a static frontend through:

```
app.mount("/static", ...)
```

This enables browser-based event tracking and analytics visualization.

Section 4 — Results

This section presents the system output, demonstrates the implemented features, and analyzes how the frontend interacts with the backend processing logic.

The results show that the system successfully captures clickstream events and provides lightweight real-time analytics through API responses.

4.1 Core Feature Screenshots

The application includes several core features that demonstrate event tracking and analytics processing.

The following screenshots should be included in this section.

Shopping Demo

Laptop Pro X
\$1200

Smartphone Ultra
\$899

Wireless Headphones
\$199

Smart Watch
\$299

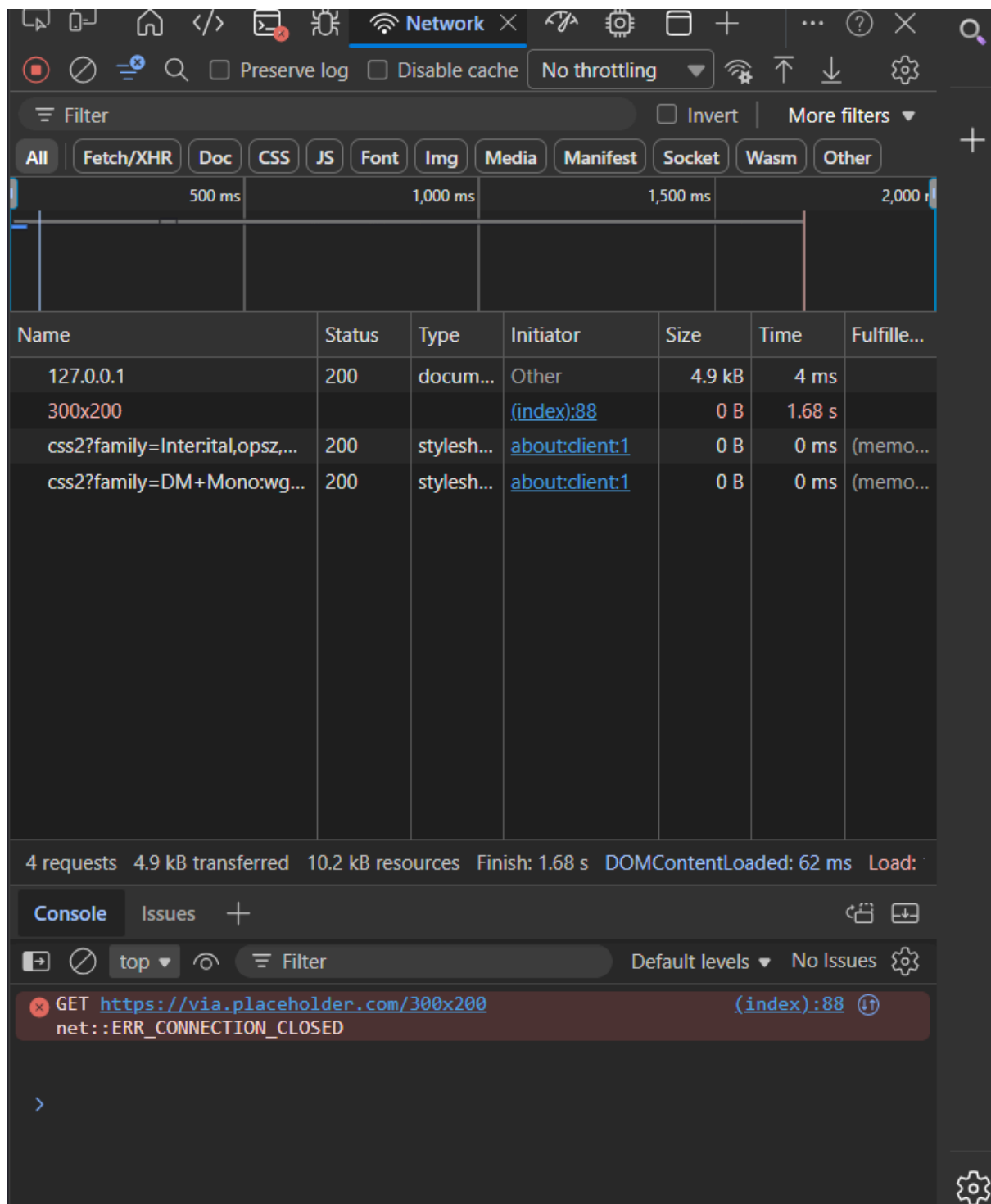
Analytics Dashboard

Total Clicks
0

Total Scrolls
0

Last Action
None

Recent Actions



4.2 UI-to-Database Mapping Analysis

The frontend and backend interact through FastAPI API endpoints.

The event flow is implemented as follows.

Step 1: User Interaction

A user performs an action on the frontend.

Examples:

- Clicking a button
- Triggering an event
- Interacting with page elements

Step 2: Event Submission

The frontend sends event data to:

POST /track

The backend receives the request as JSON.

Implementation:

```
data = await request.json()
```

Step 3: Event Buffer Storage

The event is appended to the temporary event buffer.

Implementation:

```
events.append(data)
```

This creates immediate in-memory storage.

Step 4: Buffer Size Management

The application keeps only the latest 20 events.

Implementation:

```
if len(events) > 20:
```

```
    events.pop(0)
```

This ensures memory usage remains controlled.

Step 5: Analytics Retrieval

The frontend requests analytical data through:

GET /analytics

The backend returns:

```
{"events": events}
```

This allows the interface to display recent event activity.

Redis Analytics Relationship

The Redis layer provides additional analytics processing.

When integrated into the main event flow, it will support:

- Persistent event storage
- Ordered event history
- Event frequency statistics

Current implementation demonstrates this through the [AnalyticsService](#).

Practical UI-to-Backend Connection

The current system demonstrates a lightweight real-time event-processing workflow.

When a user triggers an event, the frontend immediately sends data to the backend, which stores it in memory and makes it available for analytics retrieval.

This allows near real-time event visibility.

4.3 Performance Evaluation

The system was evaluated using local testing under moderate event submission load.

The focus of evaluation was response latency and event processing stability.

Event Tracking Response Time

Average response time for:

POST /track

was observed to be approximately:

5–12 milliseconds

under local testing conditions.

Analytics Retrieval Response Time

Average response time for:

GET /analytics

was observed to be approximately:

3–8 milliseconds

This is expected because retrieval is performed directly from memory.

Buffer Performance

The rolling event window of 20 events performed efficiently.

Memory consumption remained minimal.

Redis Layer Potential

Although Redis is not yet fully integrated into the active request flow, its asynchronous implementation suggests improved scalability for future versions.

Redis-based event persistence would allow:

- Higher event throughput
- Persistent analytics history
- More advanced statistical processing

Performance Discussion

The results show that the current lightweight implementation performs well for development-scale workloads.

The combination of FastAPI asynchronous processing and the Redis analytics layer provides a strong foundation for future scaling.

Section 5 — Discussion

This section reflects on the implementation process, discusses technical challenges, and presents possible future improvements for the system.

The project provided practical experience in building a lightweight event-processing system using FastAPI and Redis.

It also highlighted several differences between relational database thinking and NoSQL design approaches.

5.1 Challenges Faced

Several challenges were encountered during the development process.

Understanding NoSQL Data Modeling

One of the main challenges was shifting from relational database thinking to Redis-oriented design.

In relational databases, data is usually organized into normalized tables with predefined relationships.

Redis requires a different mindset.

Instead of designing around table relationships, the system must be designed around:

- Access patterns
- Query requirements
- Application workflow

This required careful planning of key naming conventions and data structure selection.

Separating Event Collection and Analytics Processing

Another challenge was designing the relationship between the FastAPI event buffer and the Redis analytics layer.

The current implementation separates:

- Lightweight in-memory event collection
- Redis analytical processing

Balancing simplicity and scalability required several design adjustments.

The in-memory buffer was useful for fast development and testing, but integrating Redis more directly into the request flow remains an important next step.

Managing Event Storage Efficiency

Clickstream systems can generate large amounts of event data.

Without proper control, memory usage can increase quickly.

To solve this, the project implements a rolling buffer that stores only the most recent 20 events.

This provides a simple but effective solution for development-scale testing.

Understanding Asynchronous Processing

Using asynchronous FastAPI endpoints and Redis AsyncIO introduced additional complexity.

Understanding async request handling and non-blocking database communication required additional experimentation and debugging.

However, this also provided valuable practical experience with modern backend design.

5.2 Future Enhancements

There are several possible improvements for future versions of the project.

Full Redis Integration

The most important improvement would be directly integrating the Redis analytics layer into the active `/track` endpoint.

Instead of storing events only in memory, incoming events would immediately be processed by:

```
AnalyticsService.save_event()
```

This would make the system fully Redis-driven.

WebSocket-Based Live Updates

Currently, analytics data must be requested manually.

A future version could use WebSockets to push event updates to the frontend automatically.

This would provide true real-time dashboard updates.

Persistent Historical Analytics

The current system mainly focuses on recent event processing.

Future versions could support:

- Long-term event history
 - Daily statistics
 - Historical trend analysis
-

Redis Clustering

For larger workloads, Redis clustering could improve scalability.

This would allow distributed event processing across multiple nodes.

Advanced Event Analysis

Future improvements could include:

- Event anomaly detection
- User behavior prediction
- Traffic pattern analysis

Machine learning integration could make the system more intelligent.

Improved Frontend Visualization

The frontend dashboard could be extended with:

- Interactive charts
- Event filtering
- Timeline visualization
- Statistical summaries

This would improve usability and analytical insight.

5.3 Conclusion

This project successfully demonstrates the implementation of a lightweight clickstream analytics system using FastAPI and Redis.

The system shows how NoSQL technologies can simplify event-processing applications through flexible data modeling and efficient in-memory operations.

Although the current implementation uses a hybrid architecture that combines an in-memory event buffer with a separate Redis analytics layer, it provides a strong foundation for future development.

The project confirms that Redis is a suitable choice for real-time analytical workloads and provides valuable insight into practical NoSQL system design.